Université : Institut Supérieur de Génie-Civil et de Gestion

(ISGG)

Année académique 2026-2027

RAPPORT DE PROJET

Gestion de Centre de Formation

Conception et développement d'une application web de gestion des étudiants, formations, inscriptions et paiements

ALLOMAVO Rabelais
DJIMASSE Aldo
TONOUKOIN Ordralie
LANTOKPODE Calmus
HOUNDEFFO Arlette
BINANZON Bil-Djèf
AWE Chadrac

I. Introduction générale

Contexte

La gestion d'un centre de formation mobilise des informations nombreuses et liées entre elles : l'identité et le parcours des apprenants, le catalogue des formations proposées, les inscriptions, les encaissements et les indicateurs de pilotage. Lorsque ces données sont consignées dans des cahiers ou des tableurs dispersés, elles deviennent difficiles à retrouver, sujettes aux erreurs de saisie et impossibles à synthétiser rapidement. C'est pour répondre à ce constat que le groupe Rabelais, dans le cadre de l'année académique 2026-2027 à l'Institut Supérieur de Génie-Civil et de Gestion (ISGG), a conçu et réalisé une application web de gestion de centre de formation.

Problématique

Comment centraliser et fiabiliser le suivi des étudiants, des formations, des inscriptions et des paiements d'un centre de formation, tout en offrant à l'administration une vue synthétique de l'activité, à l'aide d'un outil simple d'utilisation, sécurisé et accessible depuis un navigateur ?

Objectifs

Objectif général. Disposer d'une application web complète permettant de piloter l'activité d'un centre de formation. **Objectifs spécifiques :**

- enregistrer les étudiants et leur attribuer un matricule unique généré automatiquement ;
- tenir à jour un catalogue de formations organisé par catégories, avec niveau, durée, prix, capacité et période ;
- gérer les inscriptions en interdisant les doublons et le dépassement de la capacité d'une formation ;
- enregistrer les paiements rattachés à une inscription, selon plusieurs modes (espèces, mobile money, virement, carte) ;
- produire un tableau de bord statistique (indicateurs clés, évolutions, répartitions, alertes) ;
- garantir l'accès réservé aux utilisateurs authentifiés et la cohérence des données.

Périmètre du projet

Le livrable comprend une interface de programmation (API) développée avec le framework Laravel 12, une application web monopage développée avec React 19, une base de données relationnelle MySQL nommée `gestion_formation` dont un export SQL est fourni, un jeu de tests automatisés, ainsi qu'un fichier README décrivant l'installation, l'API et le dépannage. L'interface comme la base de données sont entièrement rédigées en français.

Démarche méthodologique

Le projet a été conduit de manière itérative en quatre phases : **(1)** analyse des besoins et conception du modèle de données ; **(2)** développement de l'API REST et de ses règles de gestion ; **(3)** développement de l'interface, domaine fonctionnel par domaine fonctionnel ; **(4)** tests, documentation et préparation de la mise en production. Le code est versionné avec Git. Les migrations de la base de données sont datées du 28 octobre 2026 et l'export SQL final du 1er novembre 2026.

Annonce du plan

Ce rapport présente successivement l'organisation du groupe (II), l'analyse et la conception (III), l'architecture technique (IV), la réalisation de l'API (V) et de l'interface (VI), puis les tests, l'installation et les perspectives (VII).

II. Organisation du groupe

Le projet a été réalisé par un groupe de cinq étudiants de l'ISGG. La direction du projet a été confiée à **ALLOMAVO Rabelais**, qui porte le titre de chef de groupe et de chef de projet.

Nom et prénom	Fonction dans le groupe
ALLOMAVO Rabelais	Chef de groupe et chef de projet
TONOUKOIN Ordralie	Membre du groupe
HOUNDEFFO Arlette	Membre du groupe
BINANZON Bil-Djèf	Membre du groupe
AWE Chadrac	Membre du groupe

Le rôle du chef de projet

Le chef de projet est responsable de la réussite d'ensemble du travail. Il assure la cohérence entre la réflexion, la réalisation et la livraison. Ses responsabilités ont couvert les points suivants :

- **Cadrage** : définition du périmètre fonctionnel, des objectifs et de l'ordre de réalisation des modules ;
- **Conception** : choix de la pile technologique, définition du modèle de données et du contrat d'API (routes, formats de réponse, codes d'erreur, messages en français) ;
- **Pilotage technique** : mise en place du dépôt Git du projet, des environnements de développement (XAMPP et MySQL, PHP 8.2, Node.js) et des conventions de code ;
- **Intégration** : raccordement de l'interface à l'API, vérification des règles de gestion de bout en bout ;
- **Qualité et livraison** : relecture, exécution des tests, rédaction de la documentation d'installation et préparation du build de production.

Mode de fonctionnement

Le groupe a travaillé selon un principe simple : le contrat d'interface entre l'API et l'application web est défini en premier, puis chacun peut avancer sur sa partie sans attendre les autres. Les échanges d'API passent par une couche unique côté interface (fichier `services/api.js`), ce qui évite de disperser les appels réseau dans les écrans et facilite la maintenance. Les points d'avancement et les arbitrages ont été centralisés par le chef de projet.

Environnement et outils

Domaine	Outil / technologie	Usage
Serveur de base de données	XAMPP – MySQL / MariaDB 10.4	Hébergement local de la base `gestion_formation`
Backend	PHP 8.2, Composer 2, Laravel 12	Création et exécution de l'API
Frontend	Node.js 20+, npm, Vite 8	Compilation et serveur de développement
Versionnement	Git (branche principale `main`)	Suivi des modifications du projet
Qualité	PHPUnit, oxlint	Tests automatisés, analyse statique JavaScript

> **Remarque.** Le compte administrateur est créé automatiquement par le jeu de données initial (seeder) : aucune autre donnée de démonstration n'est insérée. Les identifiants figurent dans le fichier README du dépôt et ne sont volontairement pas reproduits dans ce rapport.

III. Analyse et conception

Besoins fonctionnels

Module	Besoins couverts
Authentification	Connexion par e-mail et mot de passe, consultation de l'utilisateur connecté, déconnexion avec révocation du jeton.
Étudiants	Création, consultation, modification, suppression réversible, restauration, recherche multicritère, filtres, export CSV, photo.
Formations	Gestion du catalogue : code, intitulé, catégorie, niveau, durée, prix, capacité, dates, statut.
Catégories	Regroupement thématique des formations avec couleur d'identification.
Inscriptions	Rattachement d'un étudiant à une formation, statut, note finale, observations.
Paiements	Encaissement lié à une inscription, mode de règlement, référence, date.
Statistiques	Indicateurs clés, évolutions mensuelles, répartitions, taux de remplissage, alertes.

Règles de gestion

- **RG1** – Le matricule d'un étudiant est unique et généré au format `ETU-AAAA-NNNN` (année et séquence).
- **RG2** – L'adresse e-mail d'un étudiant et le code d'une formation sont uniques.
- **RG3** – Un étudiant ne peut être inscrit qu'une seule fois à une même formation.
- **RG4** – Le nombre d'inscriptions « en attente » ou « validée » ne peut pas dépasser la capacité maximale de la formation.
- **RG5** – Une formation ayant des inscriptions actives ne peut pas être supprimée ; une catégorie contenant des formations non plus.
- **RG6** – Un paiement est toujours rattaché à une inscription existante et son montant est strictement positif.
- **RG7** – La suppression d'un étudiant est logique (réversible) ; celle d'une inscription entraîne la suppression de ses paiements.

Modèle conceptuel des données

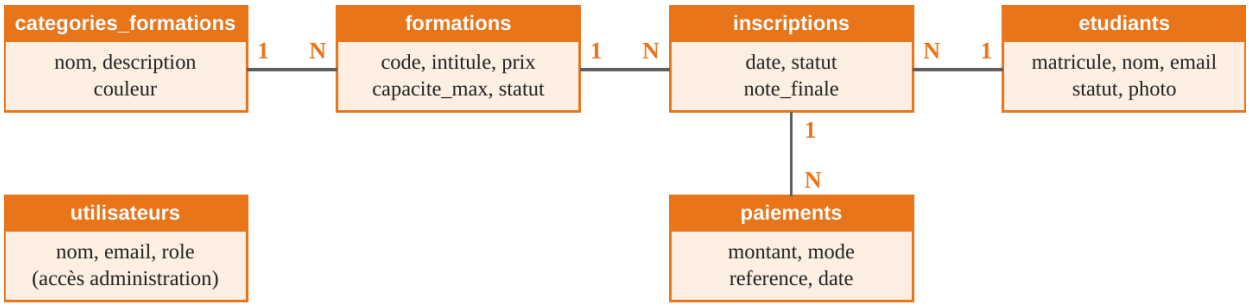


Figure 1 – Relations entre les tables (1 : un ; N : plusieurs). La table inscriptions associe étudiants et formations ; un étudiant peut suivre plusieurs formations.

IV. Architecture technique et base de données

L'application repose sur une architecture trois niveaux de type client-serveur. L'application web ne contient aucune logique d'accès aux données : elle interroge uniquement l'API REST versionnée `/api/v1`, protégée par un jeton Bearer émis par Laravel Sanctum.



Figure 2 – Architecture générale de l'application

Dictionnaire des données

Table	Colonnes principales et contraintes
utilisateurs	nom, email, mot_de_passe (haché), role, email_verified_at ; authentification de l'administration.
etudiants	matricule (unique), nom, prenom, sexe (M/F), date_naissance, lieu_naissance, email (unique), telephone, adresse, ville, pays (défaut : Bénin), photo, statut (actif, suspendu, diplome, abandon) ; suppression logique.
categories_formations	nom, description, couleur (défaut : #6366f1).
formations	code (unique), intitule, categorie_formation_id, niveau (debutant, intermediaire, avance), duree_heures, prix (décimal 10,2), capacite_max, date_debut, date_fin, statut (planifiee, en_cours, terminee, annulee).
inscriptions	etudiant_id, formation_id (couple unique), date_inscription, statut (en_attente, validee, annulee, terminee), note_finale (décimal 5,2), observations ; index sur statut et date.
paiements	inscription_id, montant (décimal 10,2), mode (especes, mobile_money, virement, carte), reference, date_paiement.
personal_access_tokens	Jetons Sanctum ; tables techniques cache, jobs et sessions fournies par Laravel.

Choix technologiques et justification

Choix	Justification
Laravel 12 + Sanctum	Framework structuré (validation, ORM Eloquent, politiques) et authentification par jeton adaptée à une application monopage.
MySQL	Base relationnelle fiable, intégrité référentielle par clés étrangères, disponible via XAMPP.
React 19 + Vite 8	Composants réutilisables, rechargement instantané en développement, chargement différé (lazy loading) des pages.
TanStack Query + Axios	Gestion du cache et de l'état serveur, intercepteurs pour le jeton et les erreurs.
Tailwind CSS 4 + Radix UI	Conception rapide d'une interface cohérente, composants accessibles.
Recharts	Graphiques du tableau de bord : aires, anneaux, histogrammes.

V. Réalisation de l'API (backend)

Organisation du code

Le backend comprend 36 fichiers PHP (environ 2 460 lignes) répartis selon la séparation des responsabilités de Laravel : les **contrôleurs** (répertoire `Controllers/Api/V1`) reçoivent les requêtes ; les **Form Requests** valident les données et renvoient des messages en français ; les **Resources** formatent les réponses JSON ; les **Models** définissent les relations et les recherches ; les **Policies** centralisent les autorisations ; les **Services** portent la logique métier réutilisable ; le trait `Filtrable` mutualise le tri sécurisé et le filtrage par plage de dates.

Authentification et sécurité

- La route `POST /auth/connexion` est la seule route publique ; elle délivre un jeton Sanctum. Toutes les autres routes sont placées derrière le middleware `auth:sanctum`.
- La déconnexion révoque le jeton ; un jeton absent ou invalide renvoie le code **401**, vérifié par un test automatisé.
- Le mot de passe est stocké haché. Les entrées sont validées par des Form Requests (par exemple : date de naissance antérieure à aujourd'hui, photo de 2 Mo maximum, e-mail unique en ignorant l'enregistrement édité).
- Le tri n'accepte que des colonnes d'une **liste blanche**, ce qui empêche l'injection de colonnes arbitraires par le paramètre `sort`.

Points d'entrée de l'API

Ressource	Routes	Particularités
Étudiants	GET, POST, PUT/PATCH, DELETE /etudiants ; POST /etudiants/{id}/restaurer ; GET /etudiants-export/csv	Pagination, recherche (nom, prénom, matricule, e-mail, téléphone, ville), filtres statut, sexe, ville, âge, dates, formation ; photo en multipart ; export CSV séparé par point-virgule.
Formations	CRUD /formations	Filtres par catégorie, statut, période ; suppression bloquée si inscriptions actives.
Catégories	CRUD /categories	Compteur de formations ; suppression refusée si non vide.
Inscriptions	CRUD /inscriptions	Contrôles de capacité et de doublon à la création et à la modification.
Paiements	CRUD /paiements	Recherche par étudiant, matricule, formation ; filtres de période.
Statistiques	14 routes GET /statistiques/…	Agrégat complet du tableau de bord ou indicateurs séparés.

Règles de gestion dans le code

La classe `InscriptionService` implémente deux contrôles. **verifierCapacite** compte les inscriptions de statut « en attente » ou « validée » de la formation (en excluant l'inscription en cours de modification) et lève une exception de validation si le total atteint la capacité maximale. **verifierDoublon** recherche une inscription existante pour le couple étudiant-formation. Ces contrôles s'ajoutent à la contrainte d'unicité posée en base. Le modèle `Etudiant` génère le matricule à la création : il recherche le dernier matricule de l'année en cours et incrémente la séquence sur quatre chiffres (exemple : `ETU-2026-0001`).

Service de statistiques

La classe `StatistiquesService` (environ 390 lignes) produit : les **indicateurs clés** (total et nombre d'étudiants actifs, formations en cours, inscriptions et chiffre d'affaires de la période, taux de remplissage moyen, taux de réussite) avec une **variation par rapport à une période précédente de même durée** ; l'évolution mensuelle des inscriptions ; les répartitions par statut, sexe, âge et catégorie ; les formations les plus demandées ; les revenus mensuels ; le remplissage par formation et les alertes de formations bientôt complètes.

VI. Réalisation de l'interface (frontend)

Organisation du code

L'application monopage compte 64 fichiers JavaScript/JSX (environ 7 080 lignes). Elle est structurée en **fonctionnalités** (répertoire `features/`) : authentification, tableau de bord, étudiants, formations, inscriptions, paiements, catégories, paramètres et page d'erreur 404. Les composants partagés se répartissent en primitives d'interface (`ui/`), éléments de mise en page (`layout/`), tableau de données (`data-table/`) et composants métier (`shared/`). Les pages sont chargées à la demande et les routes sont protégées par un composant `ProtectedRoute`.

Description des écrans

Écran	Contenu et comportement
Connexion	Formulaire e-mail / mot de passe validé côté client ; message d'erreur explicite ; redirection vers le tableau de bord après authentification.
Tableau de bord	Quatre indicateurs avec variation (étudiants, inscriptions, chiffre d'affaires, formations en cours) ; évolution des inscriptions (aires) ; statuts et sexes (anneaux) ; revenus mensuels (histogramme) ; jauges de remplissage et de réussite ; top 5 des formations ; alertes de formations bientôt complètes ; dernières inscriptions et paiements ; filtre 30 jours, 3, 6 ou 12 mois.
Étudiants	Liste paginée côté serveur, tri, recherche, filtres avancés, visibilité des colonnes, actions groupées, export CSV ; formulaire en quatre étapes (identité, coordonnées, photo, inscription) avec barre de progression, masque de téléphone et recadrage de la photo ; fiche latérale à onglets Profil, Inscriptions, Paiements.
Formations	Catalogue avec jauge de remplissage et statut ; formulaire ; fiche détaillée (revenus encaissés, moyenne des notes, liste des inscrits).
Inscriptions	Liste, création, modification ; sélection de l'étudiant et de la formation ; statut, note finale, observations ; affichage des erreurs de capacité et de doublon renvoyées par l'API.
Paiements	Enregistrement avec recherche de l'inscription (étudiant, matricule, formation) ; mode de règlement, référence, date.
Catégories	Cartes colorées avec nombre de formations ; formulaire de création et de modification.
Paramètres	Préférences de l'application, dont le thème.

Expérience utilisateur

- **Thème clair ou sombre** mémorisé, barre latérale rétractable dont l'état est conservé, interface adaptée du mobile au poste de bureau.
- **Recherche globale** par le raccourci `Ctrl + K` (étudiants, formations, inscriptions), fil d'Ariane et menu de profil.
- **Retours à l'utilisateur** : notifications (toasts) avec bouton « Annuler » après suppression, boîtes de confirmation, états vides et squelettes de chargement.
- **Protection contre la perte de données** à la fermeture d'un formulaire modifié.

Gestion des données et des erreurs

Les appels réseau passent par un client Axios unique. Un intercepteur de requête ajoute le jeton Bearer ; un intercepteur de réponse normalise les erreurs (message, détail des champs, code) et gère l'expiration de session : le jeton n'est effacé sur une erreur 401 que si la requête fautive utilisait bien le jeton actuellement stocké, ce qui évite les déconnexions intempestives après une nouvelle connexion. Les formulaires combinent React Hook Form et des schémas Zod (par exemple : nom de 2 à 100 caractères, durée de formation de 1 à 1 000 heures, capacité de 1 à 500 places) ; ces règles reprennent celles du serveur pour un retour immédiat. TanStack Query met en cache les listes et invalide les données concernées après chaque modification, et une temporisation (debounce) limite le nombre de requêtes lors de la saisie dans la recherche.

VII. Tests, mise en service et conclusion

Tests automatisés

Le backend est couvert par **42 tests fonctionnels** répartis en sept classes, exécutés avec `php artisan test` sur une base SQLite en mémoire : l'authentification (connexion valide et invalide, déconnexion, accès sans jeton refusé), les étudiants (création, matricule automatique, erreurs de validation, mise à jour, suppression, recherche, envoi de photo, export CSV), les formations (suppression autorisée ou bloquée), les catégories (unicité du nom, suppression refusée si non vide), les inscriptions (doublon refusé, capacité dépassée refusée), les paiements (création, validation) et les statistiques (indicateurs, évolution, répartition, remplissage, revenus, agrégat du tableau de bord).

Installation et mise en service

```
1. Démarrer MySQL (XAMPP) et créer la base gestion_formation (utf8mb4_unicode_ci)
2. cd backend ; composer install ; copier .env.example en .env ; php artisan key:generate
3. php artisan migrate:fresh --seed ; php artisan storage:link ; php artisan serve --port=8000
4. cd frontend ; npm install ; copier .env.example en .env ; npm run dev
(http://localhost:5173)
```

Pour la production, la commande `npm run build` génère le dossier `frontend/dist`, servi par Apache ou Nginx avec redirection de `/api` vers Laravel ; côté API, on exécute `config:cache`, `route:cache` et `view:cache` avec `APP_ENV=production` et `APP_DEBUG=false`. Le README fournit un tableau de dépannage (MySQL non démarré, base inexistante, jetons supprimés après un `migrate:fresh`, photos non affichées, port occupé).

Difficultés rencontrées

- **Gestion de la session** : éviter qu'une erreur 401 tardive n'invalide un jeton tout juste obtenu, résolu par la comparaison des jetons dans l'intercepteur Axios.
- **Cohérence des inscriptions** : empêcher à la fois les doublons et le dépassement de capacité, y compris lors de la modification d'une inscription existante, résolu par le service dédié et la contrainte d'unicité en base.
- **Volume de données affichées** : pagination serveur, filtres combinables et temporisation de la recherche.

Conclusion

Le projet a permis de livrer une application de gestion de centre de formation complète, qui couvre le parcours d'un apprenant, de la création de son dossier à l'encaissement de ses frais, et qui offre à l'administration un tableau de bord de pilotage. Les objectifs fixés ont été atteints : données centralisées, règles de gestion protégées côté serveur et en base, accès sécurisé et interface moderne. Il a aussi permis au groupe, sous la conduite de son chef de projet ALLOMAVO Rabelais, de mettre en pratique la conception d'une API REST, le développement d'une application monopage, les tests automatisés et le travail collaboratif.

Perspectives

- gestion de profils d'utilisateurs (administrateur, secrétariat, formateur) à partir du champ `role` déjà présent ;
- génération de reçus de paiement et d'attestations de formation au format PDF ;
- notifications par e-mail ou SMS (formation bientôt complète, rappel de paiement) ;
- paiement en ligne par mobile money avec rapprochement automatique des références ;
- intégration continue et déploiement sur un serveur de production ; espace dédié aux étudiants.